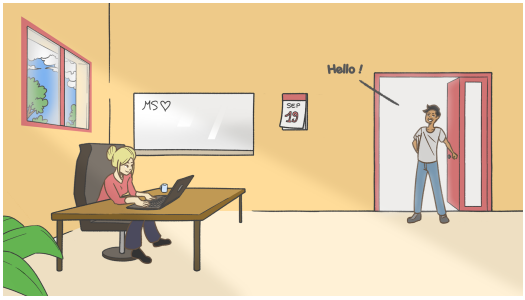
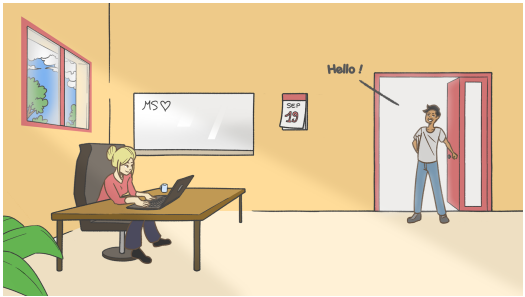
The Trichotomy of Property Testing Regular Languages

Gabriel Bathie Nathanaël Fijalkow Corto Mascle

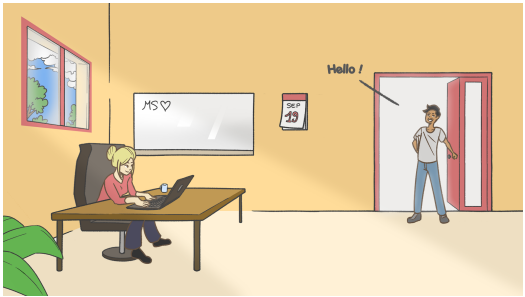
LaBRI, Bordeaux MPI-SWS Kaiserslautern

Algorithms & Complexity seminar, Oxford, 2026

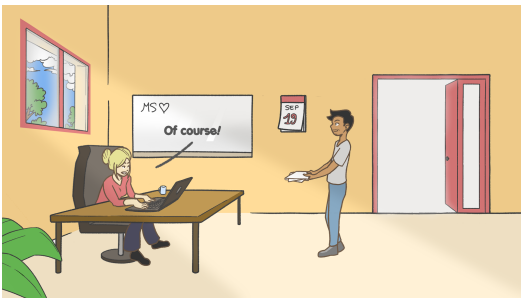

MS♡
SEP
13
Hello !

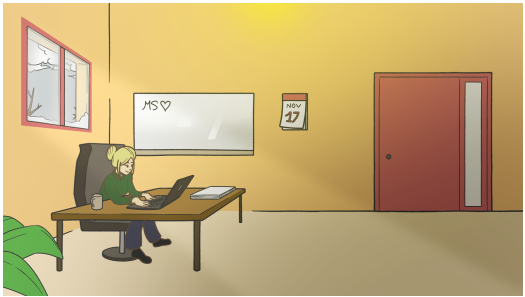

MS ♡
Hello !

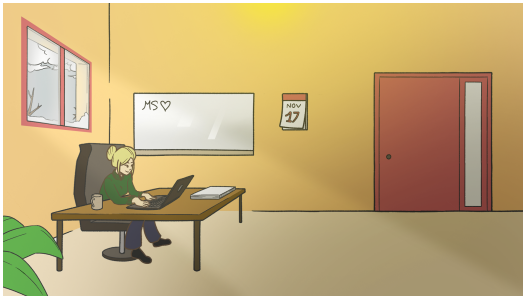

MS ♡
Can you review my manuscript?

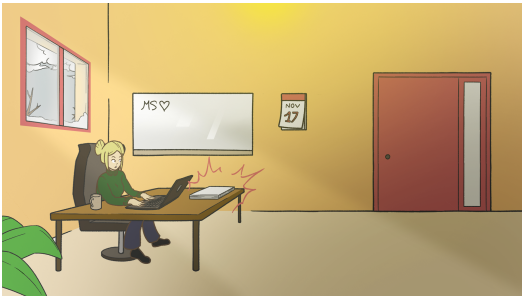

MS♡
Sep
13
Hello !

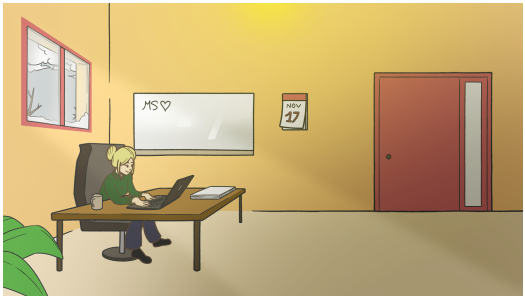

MS♡
Sep
13
Can you review my manuscript?

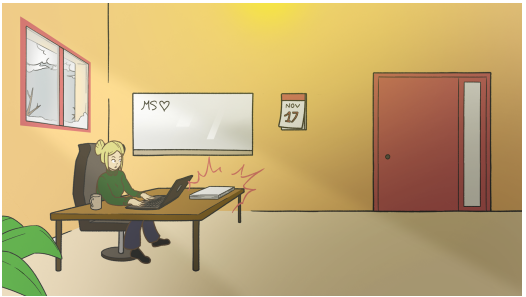

MS♡
Of course!
Sep
13

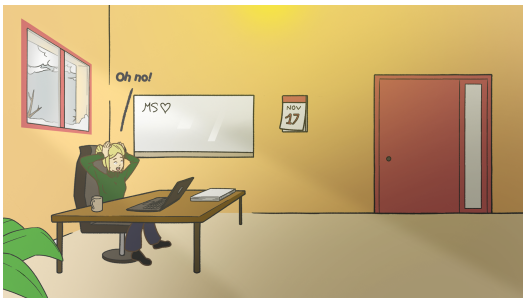

MS♡
NOV
17

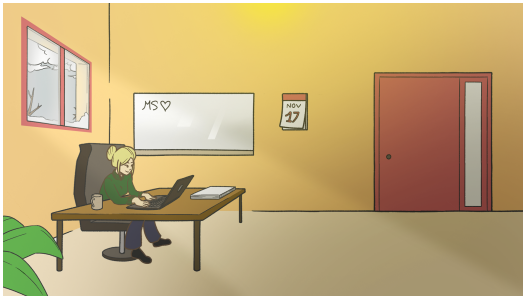

MS♡
NOV
17

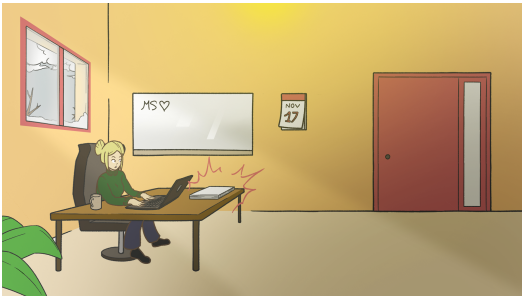

MS♡
NOV
17

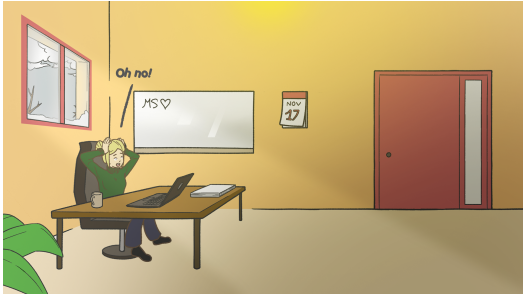

MS♡
NOV
17

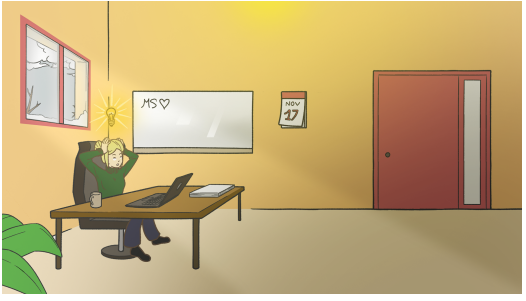

MS♡
NOV
17

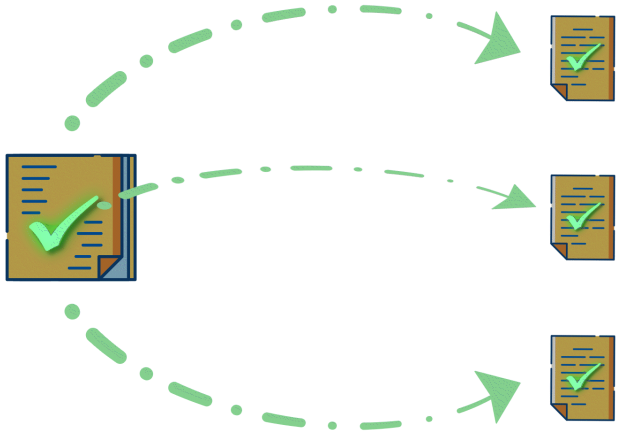

Oh no!
MS♡
NOV
17

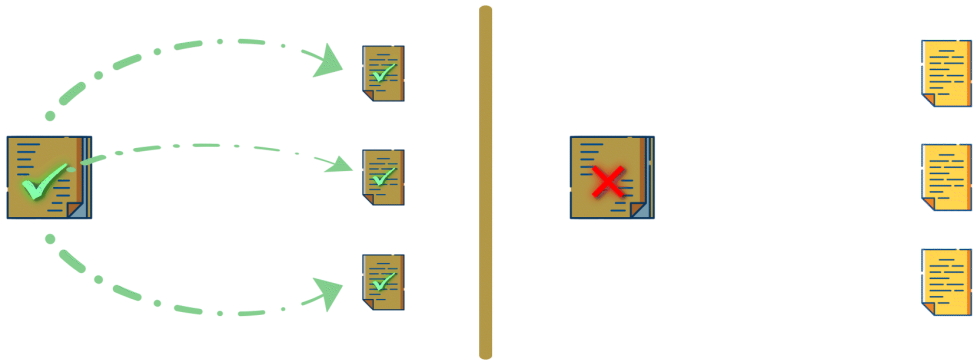

MS♡
NOV
17

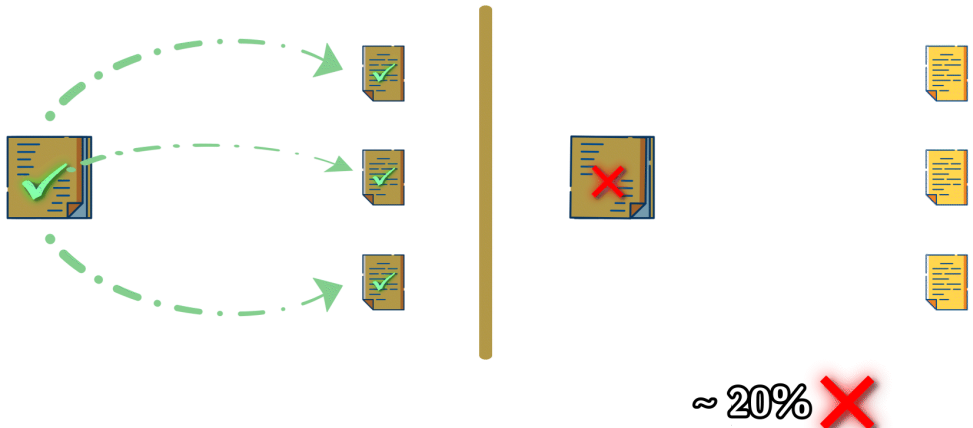

MS♡
NOV
17

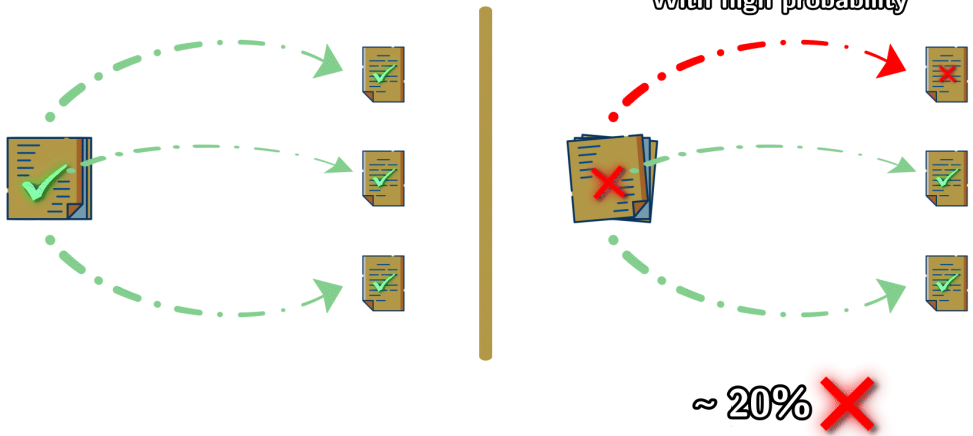

Oh no!
MS♡
NOV
17


MS♡
NOV
17






~ 20%


With high probability
~ 20%

Property testing

Class of objects $\mathcal{C}$

Ex: words, trees, graphs, lists, matrices

Size function $|.|$

Ex: number of vertices+edges on graphs

Distance function d

Ex: edge-edit distance on graphs

Subclass P

Ex: connected graphs, k-colourable graphs

Property testing

Class of objects $\mathcal{C}$

Ex: words, trees, graphs, lists, matrices

Size function $|.|$

Ex: number of vertices+edges on graphs

Distance function d

Ex: edge-edit distance on graphs

Subclass P

Ex: connected graphs, k-colourable graphs

Fix a parameter $\varepsilon > 0$.

Property testing

Class of objects $\mathcal{C}$

Ex: words, trees, graphs, lists, matrices

Size function $|.|$

Ex: number of vertices+edges on graphs

Distance function d

Ex: edge-edit distance on graphs

Subclass P

Ex: connected graphs, k-colourable graphs

Fix a parameter $\varepsilon > 0$.

Property testing

Input: $obj \in \mathcal{C}$

Output: If $obj \in P \rightarrow$ **Yes**

If $d(obj, P) > \varepsilon |obj| \rightarrow$ **No** with probability $\geq 1/3$

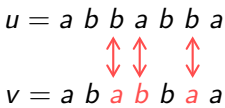
Example

Words over $\{a, b\}$, equipped with the *Hamming distance*.

Example

Words over $\{a, b\}$, equipped with the *Hamming distance*.

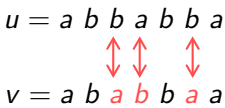
$$d(u,v) = \begin{cases} |\{i \mid u[i] \neq v[i]\}| \text{ if } |u| = |v| \\ +\infty \text{ if } |u| \neq |v| \end{cases}$$



Example

Words over $\{a, b\}$, equipped with the *Hamming distance*.

$$d(u,v) = \begin{cases} |\{i \mid u[i] \neq v[i]\}| \text{ if } |u| = |v| \\ +\infty \text{ if } |u| \neq |v| \end{cases}$$



Property $P = a^*$

Example

Words over $\{a, b\}$, equipped with the *Hamming distance*.

$$d(u,v) = \begin{cases} |\{i \mid u[i] \neq v[i]\}| \text{ if } |u| = |v| \\ +\infty \text{ if } |u| \neq |v| \end{cases}$$

$u = a\ b\ b\ a\ b\ b\ a$

$v = a\ b\ a\ b\ b\ a\ a$

Property $P = a^*$

Find an algorithm such that, on a word w,
If $w \in a^* \rightarrow$ **Yes**
If $d(w, a^*) > \varepsilon|w| \rightarrow$ **No** with constant positive probability

Example

Words over $\{a, b\}$, equipped with the *Hamming distance*.

$$d(u,v) = \begin{cases} |\{i \mid u[i] \neq v[i]\}| \text{ if } |u| = |v| \\ +\infty \text{ if } |u| \neq |v| \end{cases}$$

$u = a\ b\ b\ a\ b\ b\ a$

$v = a\ b\ a\ b\ b\ a\ a$

Property $P = a^*$

Find an algorithm such that, on a word w,
If $w \in a^* \rightarrow$ **Yes**
If $d(w, a^*) > \varepsilon|w| \rightarrow$ **No** with constant positive probability

Sample $1/\varepsilon$ letters at random, answer yes iff we find no b.

independent of $|w|$

Example

Words over $\{a, b\}$, equipped with the *Hamming distance*.

$$d(u,v) = \begin{cases} |\{i \mid u[i] \neq v[i]\}| \text{ if } |u| = |v| \\ +\infty \text{ if } |u| \neq |v| \end{cases}$$

$u = a\ b\ b\ a\ b\ b\ a$

$v = a\ b\ a\ b\ b\ a\ a$

Property $P = a^*$

Find an algorithm such that, on a word w,
If $w \in a^* \rightarrow$ **Yes**
If $d(w, a^*) > \varepsilon|w| \rightarrow$ **No** with constant positive probability

Sample $1/\varepsilon$ letters at random, answer yes iff we find no b.

independent of $|w|$

Example

Words over $\{a, b\}$, equipped with the *Hamming distance*.

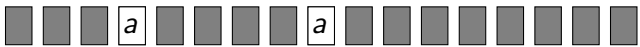
$$d(u,v) = \begin{cases} |\{i \mid u[i] \neq v[i]\}| \text{ if } |u| = |v| \\ +\infty \text{ if } |u| \neq |v| \end{cases}$$

$u = a\ b\ b\ a\ b\ b\ a$

$v = a\ b\ a\ b\ b\ a\ a$

Property $P = a^*$

Find an algorithm such that, on a word w,
If $w \in a^* \rightarrow$ **Yes**
If $d(w, a^*) > \varepsilon |w| \rightarrow$ **No** with constant positive probability



Sample $1/\varepsilon$ letters at random, answer yes iff we find no b.

independent of $|w|$

Example

Words over $\{a, b\}$, equipped with the *Hamming distance*.

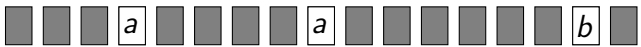
$$d(u,v) = \begin{cases} |\{i \mid u[i] \neq v[i]\}| \text{ if } |u| = |v| \\ +\infty \text{ if } |u| \neq |v| \end{cases}$$

$u = a\ b\ b\ a\ b\ b\ a$

$v = a\ b\ a\ b\ b\ a\ a$

Property $P = a^*$

Find an algorithm such that, on a word w,
If $w \in a^* \rightarrow$ **Yes**
If $d(w, a^*) > \varepsilon|w| \rightarrow$ **No** with constant positive probability



Sample $\underbrace{1/\varepsilon}$ letters at random, answer yes iff we find no b.

independent of $|w|$

Example

Words over $\{a, b\}$, equipped with the *Hamming distance*.

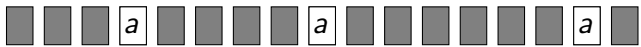
$$d(u,v) = \begin{cases} |\{i \mid u[i] \neq v[i]\}| \text{ if } |u| = |v| \\ +\infty \text{ if } |u| \neq |v| \end{cases}$$

$u = a\ b\ b\ a\ b\ b\ a$

$v = a\ b\ a\ b\ b\ a\ a$

Property $P = a^*$

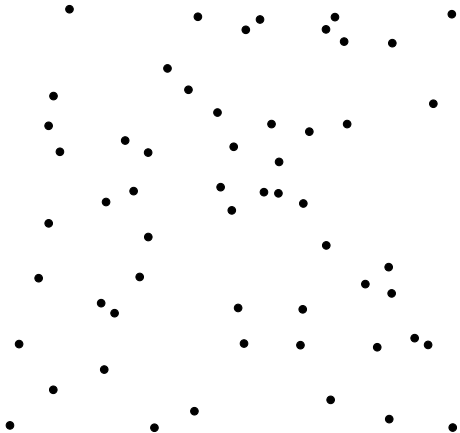
Find an algorithm such that, on a word w,
If $w \in a^* \rightarrow$ **Yes**
If $d(w, a^*) > \varepsilon |w| \rightarrow$ **No** with constant positive probability



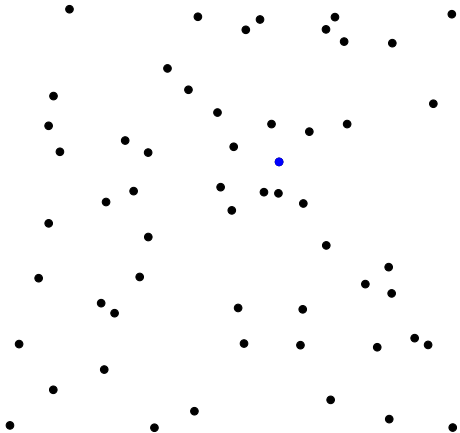
Sample $1/\varepsilon$ letters at random, answer yes iff we find no b.

independent of $|w|$

Example: connectivity

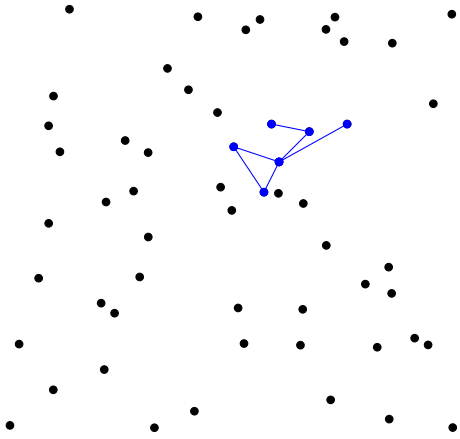


Example: connectivity



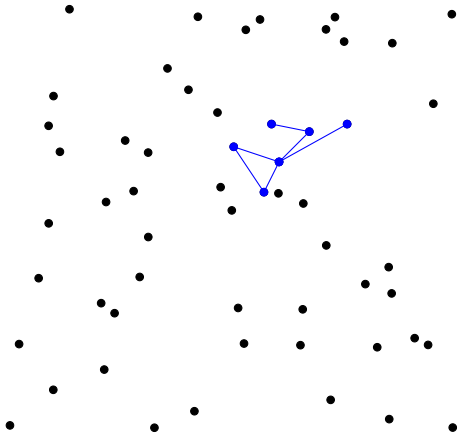
- Pick a random vertex

Example: connectivity



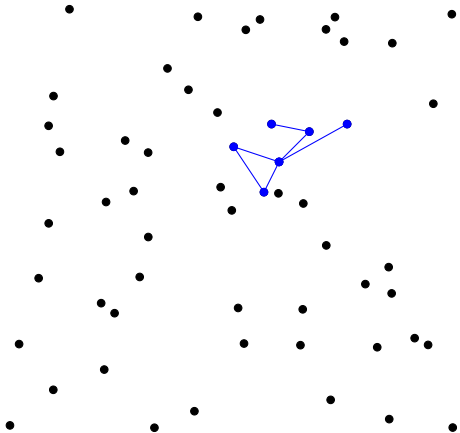
- Pick a random vertex
- Explore its connected component by DFS

Example: connectivity



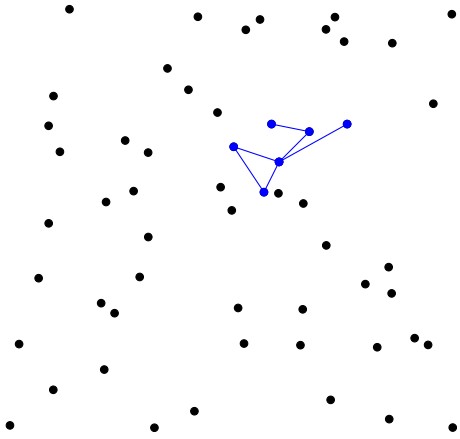
- Pick a random vertex
- Explore its connected component by DFS
- If it has size $< 2/\varepsilon$ then return No

Example: connectivity



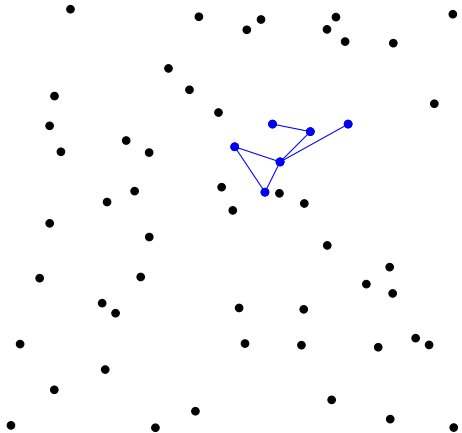
- Pick a random vertex
- Explore its connected component by DFS
- If it has size $< 2/\varepsilon$ then return No
- Otherwise pick another point etc...
- Return Yes if $> 1/\varepsilon$ points have been picked

Example: connectivity



for $0 \leq t \leq 1/\varepsilon$ **do**
    $v \leftarrow rand(V)$
    if $|SCC(v)| < 2/\varepsilon$ **then**
        return No
return Yes

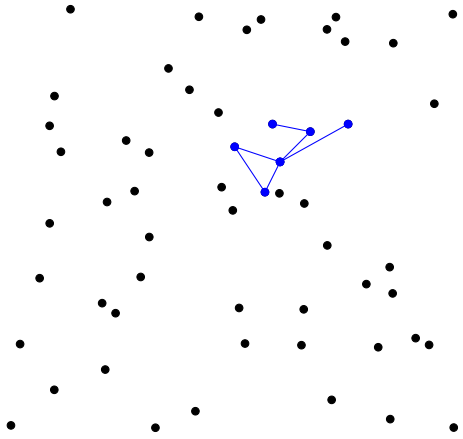
Example: connectivity



for $0 \leq t \leq 1/\varepsilon$ **do**
  $v \leftarrow rand(V)$
  if $|SCC(v)| < 2/\varepsilon$ **then**
    return No
return Yes

- G is connected $\Rightarrow$ Yes.
- $d(G, P_{connected}) \geq \varepsilon|V|$
  $\Rightarrow$ G has $\geq \varepsilon|V|$ connected components
  $\Rightarrow$ G has $\geq \varepsilon|V|/2$ connected components of size $\leq 2/\varepsilon$
  $\Rightarrow$ No with constant probability.

Example: connectivity



for $0 \leq t \leq 1/\varepsilon$ **do**
  $v \leftarrow rand(V)$
  if $|SCC(v)| < 2/\varepsilon$ **then**
    return No
return Yes

- G is connected $\Rightarrow$ Yes.
- $d(G, P_{connected}) \geq \varepsilon|V|$
  $\Rightarrow$ G has $\geq \varepsilon|V|$ connected components
  $\Rightarrow$ G has $\geq \varepsilon|V|/2$ connected components of size $\leq 2/\varepsilon$
  $\Rightarrow$ No with constant probability.

$\mathcal{O}(1/\varepsilon^2)$ queries.

Back to words

Property testing on words

Input: $w \in \Sigma^*$

Output: If $w \in L \rightarrow$ **Yes**

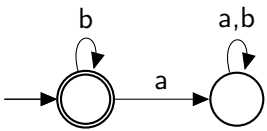
If $d(w, L) > \varepsilon|w| \rightarrow$ **No** with probability $\geq 1/3$

Problem

Find efficient property testing algorithms for regular languages.

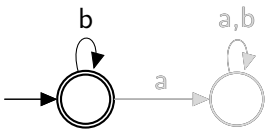
Example

$L = b^*$ over $\Sigma = \{a, b\}$



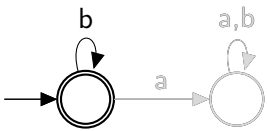
Example

$L = b^*$ over $\Sigma = \{a, b\}$



Example

$L = b^*$ over $\Sigma = \{a, b\}$



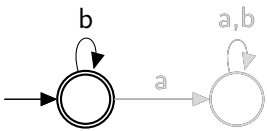
for $1 \leq k \leq 1/\varepsilon$ **do**
  $i \leftarrow rand(\{1, \ldots, |w|\})$
  if $w[i] = a$ **then**
    return No
return Yes

$\mathcal{O}(1/\varepsilon)$ queries

- $w \in L \Rightarrow$ **Yes**
- $d(w, L) > \varepsilon|w| \Rightarrow \mathbb{P}(\textbf{No}) \geq 1 - (1 - \varepsilon)^{1/\varepsilon} \geq 1/3$

Example

$L = b^*$ over $\Sigma = \{a, b\}$



for $1 \leq k \leq 1/\varepsilon$ **do**
  $i \leftarrow rand(\{1, \ldots, |w|\})$
  if $w[i] = a$ **then**
    return No
return Yes

$\mathcal{O}(1/\varepsilon)$ queries

- $w \in L \Rightarrow$ **Yes**
- $d(w, L) > \varepsilon|w| \Rightarrow \mathbb{P}(\textbf{No}) \geq 1 - (1 - \varepsilon)^{1/\varepsilon} \geq 1/3$

Blocking factor = Word that is not a factor of any word in L.

$$\mathcal{BF}((a^+bb)^*a^*) = \Sigma^*(aba + bbb)\Sigma^*$$

[1] Alon, Krivelevich, Newman, Szegedy 2001

Blocking factor = Word that is not a factor of any word in L.

$$\mathcal{BF}((a^+bb)^*a^*) = \Sigma^*(aba + bbb)\Sigma^*$$

Theorem[1]

If L is recognised by a **strongly connected** automaton, then for all w,

$$d(w, L(\mathcal{A})) \geq \varepsilon|w| \Rightarrow w \text{ contains } \Omega(\varepsilon|w|) \text{ disjoint blocking factors.}$$

[1] Alon, Krivelevich, Newman, Szegedy 2001

Blocking factor = Word that is not a factor of any word in L.

$$\mathcal{BF}((a^+bb)^*a^*) = \Sigma^*(aba + bbb)\Sigma^*$$

Theorem[1]

If L is recognised by a **strongly connected** automaton, then for all w,

$$d(w, L(\mathcal{A})) \geq \varepsilon|w| \Rightarrow w \text{ contains } \Omega(\varepsilon|w|) \text{ disjoint blocking factors.}$$

$w =$ 

Blocking factor = Word that is not a factor of any word in L.

$$\mathcal{BF}((a^+bb)^*a^*) = \Sigma^*(aba + bbb)\Sigma^*$$

Theorem[1]

If L is recognised by a **strongly connected** automaton, then for all w,

$$d(w, L(\mathcal{A})) \geq \varepsilon|w| \Rightarrow w \text{ contains } \Omega(\varepsilon|w|) \text{ disjoint blocking factors.}$$

$w =$ 

[1] Alon, Krivelevich, Newman, Szegedy 2001

Blocking factor = Word that is not a factor of any word in L.

$$\mathcal{BF}((a^+bb)^*a^*) = \Sigma^*(aba + bbb)\Sigma^*$$

Theorem[1]

If L is recognised by a **strongly connected** automaton, then for all w,

$$d(w, L(\mathcal{A})) \geq \varepsilon|w| \Rightarrow w \text{ contains } \Omega(\varepsilon|w|) \text{ disjoint blocking factors.}$$

$w =$ 

[1] Alon, Krivelevich, Newman, Szegedy 2001

Blocking factor = Word that is not a factor of any word in L.

$$\mathcal{BF}((a^+bb)^*a^*) = \Sigma^*(aba + bbb)\Sigma^*$$

Theorem[1]

If L is recognised by a **strongly connected** automaton, then for all w,

$$d(w, L(\mathcal{A})) \geq \varepsilon|w| \Rightarrow w \text{ contains } \Omega(\varepsilon|w|) \text{ disjoint blocking factors.}$$

$w =$ 

[1] Alon, Krivelevich, Newman, Szegedy 2001

Blocking factor = Word that is not a factor of any word in L.

$$\mathcal{BF}((a^+bb)^*a^*) = \Sigma^*(aba + bbb)\Sigma^*$$

Theorem[1]

If L is recognised by a **strongly connected** automaton, then for all w,

$$d(w, L(\mathcal{A})) \geq \varepsilon|w| \Rightarrow w \text{ contains } \Omega(\varepsilon|w|) \text{ disjoint blocking factors.}$$

$w =$ 

[1] Alon, Krivelevich, Newman, Szegedy 2001

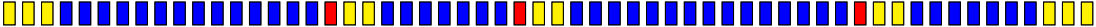
Blocking factor = Word that is not a factor of any word in L.

$$\mathcal{BF}((a^+bb)^*a^*) = \Sigma^*(aba + bbb)\Sigma^*$$

Theorem[1]

If L is recognised by a **strongly connected** automaton, then for all w,

$$d(w, L(\mathcal{A})) \geq \varepsilon|w| \Rightarrow w \text{ contains } \Omega(\varepsilon|w|) \text{ disjoint blocking factors.}$$

$w =$ 

[1] Alon, Krivelevich, Newman, Szegedy 2001

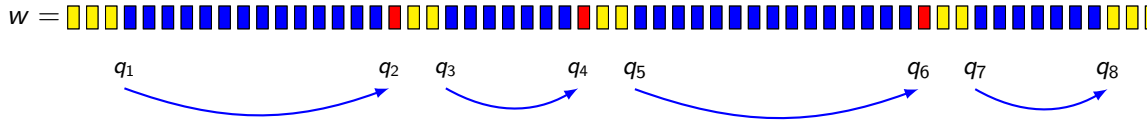
Blocking factor = Word that is not a factor of any word in L.

$$\mathcal{BF}((a^+bb)^*a^*) = \Sigma^*(aba + bbb)\Sigma^*$$

Theorem[1]

If L is recognised by a **strongly connected** automaton, then for all w,

$$d(w, L(\mathcal{A})) \geq \varepsilon|w| \Rightarrow w \text{ contains } \Omega(\varepsilon|w|) \text{ disjoint blocking factors.}$$



[1] Alon, Krivelevich, Newman, Szegedy 2001

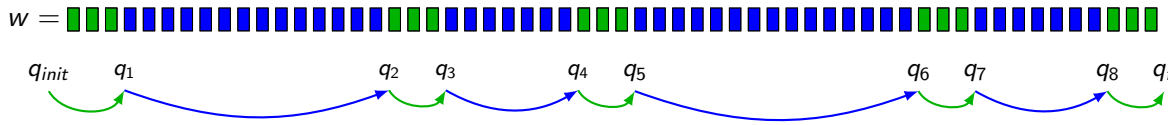
Blocking factor = Word that is not a factor of any word in L.

$$\mathcal{BF}((a^+bb)^*a^*) = \Sigma^*(aba + bbb)\Sigma^*$$

Theorem[1]

If L is recognised by a **strongly connected** automaton, then for all w,

$$d(w, L(\mathcal{A})) \geq \varepsilon|w| \Rightarrow w \text{ contains } \Omega(\varepsilon|w|) \text{ disjoint blocking factors.}$$



[1] Alon, Krivelevich, Newman, Szegedy 2001

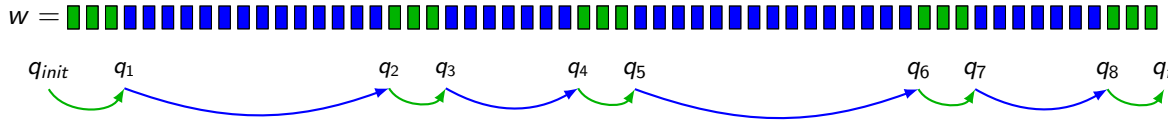
Blocking factor = Word that is not a factor of any word in L.

$$\mathcal{BF}((a^+bb)^*a^*) = \Sigma^*(aba + bbb)\Sigma^*$$

Theorem[1]

If L is recognised by a **strongly connected** automaton, then for all w,

$$d(w, L(\mathcal{A})) \geq \varepsilon|w| \Rightarrow w \text{ contains } \Omega(\varepsilon|w|) \text{ disjoint blocking factors.}$$



Theorem[1]

w has $\Omega(\varepsilon|w|)$ blocking factors $\Rightarrow$ w has $\Omega(\varepsilon|w|)$ blocking factors of size $\mathcal{O}(1/\varepsilon)$.

[1] Alon, Krivelevich, Newman, Szegedy 2001

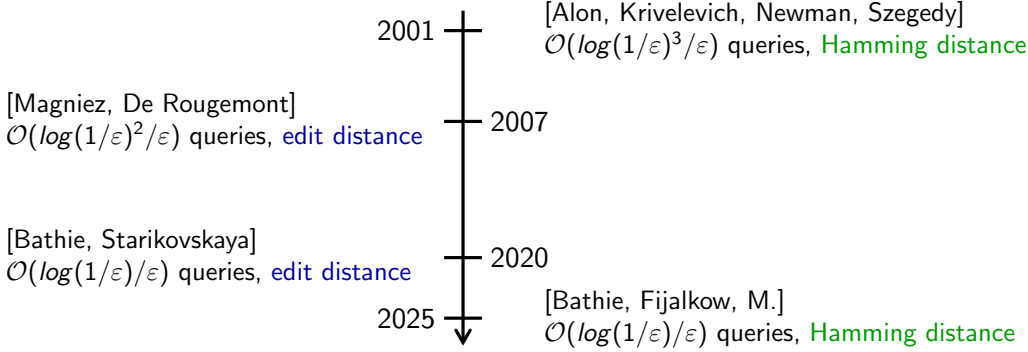
$d(w, L(\mathcal{A})) \geq \varepsilon|w| \Rightarrow w$ contains $M\varepsilon|w|$ blocking factors of size $\leq M/\varepsilon$. .

for $1 \leq k \leq M/\varepsilon$ **do**
  $i \leftarrow rand(\{1, \ldots, |w|\})$
  if $w[i - M/\varepsilon, i + M/\varepsilon] \in \mathcal{BF}(L)$ **then**
    return No
return Yes

Theorem

If L is recognised by a strongly connected automaton then it is testable with $\mathcal{O}(1/\varepsilon^2)$ queries.

Upper bounds



Theorem

Every regular language is testable using $\mathcal{O}(log(1/\varepsilon)/\varepsilon)$ queries under the Hamming distance.

for $0 \leq t \leq log(M/\varepsilon)$ **do**
  for $1 \leq k \leq 1/(\varepsilon 2^t)$ **do**
    $i \leftarrow rand(\{1, \ldots, |w|\})$
    if $w[i - 2^t, i + 2^t] \in \mathcal{BF}(L)$ **then**
      return No
return Yes

Theorem

Every regular language is testable using $\mathcal{O}(log(1/\varepsilon)/\varepsilon)$ queries under the Hamming distance.

for $0 \leq t \leq log(M/\varepsilon)$ **do**
  for $1 \leq k \leq 1/(\varepsilon 2^t)$ **do**
    $i \leftarrow rand(\{1, \ldots, |w|\})$
    if $w[i - 2^t, i + 2^t] \in \mathcal{BF}(L)$ **then**
      return No
return Yes

Assume there are many factors of length ℓ =1, 2, 4, 8, ..., $1/\varepsilon$

Theorem

Every regular language is testable using $\mathcal{O}(log(1/\varepsilon)/\varepsilon)$ queries under the Hamming distance.

for $0 \leq t \leq log(M/\varepsilon)$ **do**
  for $1 \leq k \leq 1/(\varepsilon 2^t)$ **do**
    $i \leftarrow rand(\{1, \ldots, |w|\})$
    if $w[i - 2^t, i + 2^t] \in \mathcal{BF}(L)$ **then**
      return No
return Yes

Assume there are many factors of length ℓ =1, 2, 4, 8, ..., $1/\varepsilon$

Sample $1/\varepsilon\ell$ factors of length 2ℓ

Lower bounds

Theorem[1]

The language $L = b^*$ over $\{a, b\}^*$ requires $\Omega(1/\varepsilon)$ queries.

[1] Alon, Krivelevich, Newman, Szegedy '01
[2] Bathie, Starikovskaya '21

Lower bounds

Theorem[1]

The language $L = b^*$ over $\{a, b\}^*$ requires $\Omega(1/\varepsilon)$ queries.

Theorem[2]

Some languages require $\Omega(log(1/\varepsilon)/\varepsilon)$ queries.

[1] Alon, Krivelevich, Newman, Szegedy '01
[2] Bathie, Starikovskaya '21

Lower bounds

Theorem[1]

The language $L = b^*$ over $\{a, b\}^*$ requires $\Omega(1/\varepsilon)$ queries.

Theorem[2]

Some languages require $\Omega(log(1/\varepsilon)/\varepsilon)$ queries.

Key ingredient: *Yao's minimax principle*

- Choose a distribution $\mathcal{D}$ over inputs
- Show lower bound on expected complexity of deterministic algorithms over $\mathcal{D}$.

[1] Alon, Krivelevich, Newman, Szegedy '01
[2] Bathie, Starikovskaya '21

Yao's minimax principle

Take an arbitrary distribution $\mathcal{D}$ over inputs, then for all randomised algorithm $\mathbf{R}$,

$$\min_{\mathbf{A}\ det.} \mathbb{E}_{in\sim\mathcal{D}}(cost(\mathbf{A}, in)) \leq \max_{in\in I} \mathbb{E}_{r\sim\mathcal{U}}(cost(\mathbf{R}, r, in))$$

Best expected complexity of a deterministic algorithm over $\mathcal{D}$ $\leq$ the worst-case complexity of $\mathbf{R}$

Yao's minimax principle

Take an arbitrary distribution $\mathcal{D}$ over inputs, then for all randomised algorithm $\mathbf{R}$,

$$\min_{\mathbf{A}\ det.} \mathbb{E}_{in\sim\mathcal{D}}(cost(\mathbf{A}, in)) \leq \max_{in\in I} \mathbb{E}_{r\sim\mathcal{U}}(cost(\mathbf{R}, r, in))$$

Best expected complexity of a deterministic algorithm over $\mathcal{D}$ $\leq$ the worst-case complexity of $\mathbf{R}$

If $\mathbf{R}$ uses d queries, we infer

$$\min_{\substack{\mathbf{A}\ det.\\ d \text{ queries}}} \mathbb{P}_{in\sim\mathcal{D}}(\mathbf{A} \text{ incorrect on } in) \leq \max_{in\in I} \mathbb{P}_{r\sim\mathcal{U}}(\mathbf{R} \text{ incorrect on } in)$$

Yao's minimax principle

Take an arbitrary distribution $\mathcal{D}$ over inputs, then for all randomised algorithm $\mathbf{R}$,

$$\min_{\mathbf{A}\ det.} \mathbb{E}_{in\sim\mathcal{D}}(cost(\mathbf{A}, in)) \leq \max_{in\in I} \mathbb{E}_{r\sim\mathcal{U}}(cost(\mathbf{R}, r, in))$$

Best expected complexity of a deterministic algorithm over $\mathcal{D}$ $\leq$ the worst-case complexity of $\mathbf{R}$

If $\mathbf{R}$ uses d queries, we infer

$$\min_{\substack{\mathbf{A}\ det.\\ d \text{ queries}}} \mathbb{P}_{in\sim\mathcal{D}}(\mathbf{A} \text{ incorrect on } in) \leq \max_{in\in I} \mathbb{P}_{r\sim\mathcal{U}}(\mathbf{R} \text{ incorrect on } in)$$

Every det. algo with d queries is incorrect with proba $> 1/3$ over $\mathcal{D}$

$\Rightarrow$

Every property testing algorithm needs $> d$ queries.

Yao's minimax principle

Every det. algo with d queries is incorrect with proba $> 1/3$ over $\mathcal{D}$

$\Rightarrow$

Every property testing algorithm needs $> d$ queries.

Theorem[1]

The language $L = b^*$ over $\{a, b\}^*$ requires $\Omega(1/\varepsilon)$ queries.

Yao's minimax principle

Every det. algo with d queries is incorrect with proba $> 1/3$ over $\mathcal{D}$

$\Rightarrow$

Every property testing algorithm needs $> d$ queries.

Theorem[1]

The language $L = b^*$ over $\{a, b\}^*$ requires $\Omega(1/\varepsilon)$ queries.

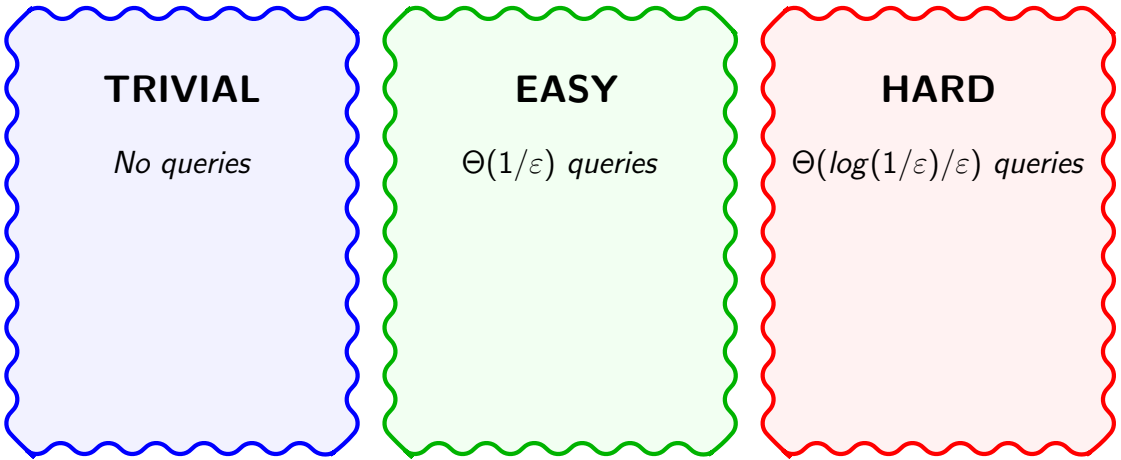
Choose $\mathcal{D}$ so that:

- b^n with proba $1/2$
- $b^{i\varepsilon/n}a^{\varepsilon/n}b^{n-(i+1)\varepsilon/n}$ with proba $\varepsilon/2$.

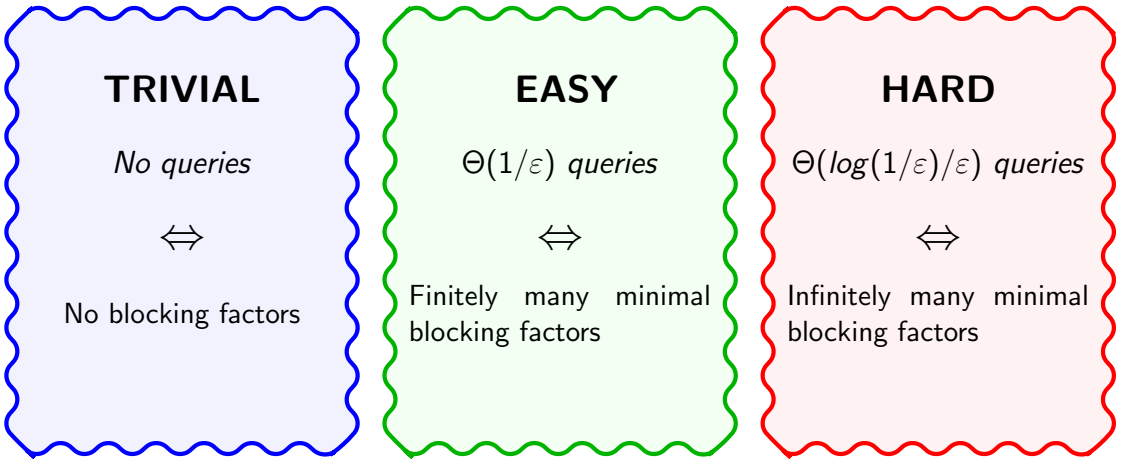
A deterministic property testing algorithm:

- must answer Yes if it only sees b
- has a probability $> 2/3$ to only see b if it queries $< \varepsilon/3$ letters.

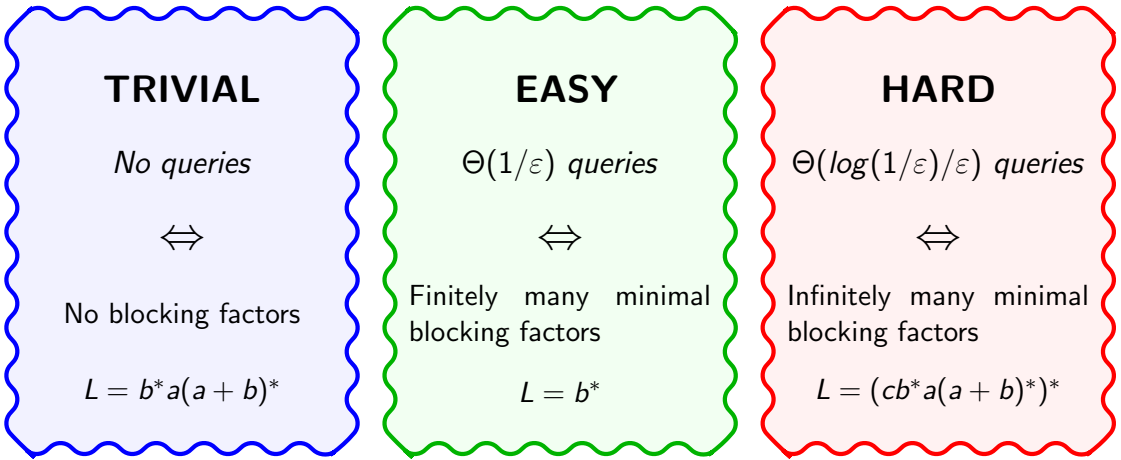
Trichotomy Theorem for strongly connected automata



Trichotomy Theorem for strongly connected automata



Trichotomy Theorem for strongly connected automata



Trichotomy Theorem for strongly connected automata

TRIVIAL

No queries

$\Leftrightarrow$

No blocking factors

$L = b^*a(a+b)^*$

EASY

$\Theta(1/\varepsilon)$ *queries*

$\Leftrightarrow$

Finitely many minimal blocking factors

$L = b^*$

HARD

$\Theta(log(1/\varepsilon)/\varepsilon)$ *queries*

$\Leftrightarrow$

Infinitely many minimal blocking factors

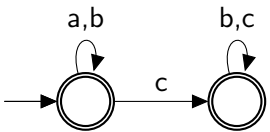
$L = (cb^*a(a+b)^*)^*$

What about automata with multiple SCCs?

Example

$L = (a + b)^*(b + c)^*$ over $\Sigma = \{a, b, c\}$.

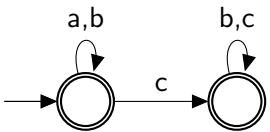
Minimal blocking factors $= cb^*a$.



Example

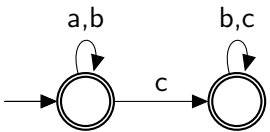
$L = (a + b)^*(b + c)^*$ over $\Sigma = \{a, b, c\}$.

Minimal blocking factors $= cb^*a$.



Example

$L = (a + b)^*(b + c)^*$ over $\Sigma = \{a, b, c\}$.

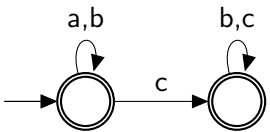


Minimal blocking factors $= cb^*a$.

$I_a, I_c \leftarrow \emptyset$
for $1 \leq k \leq 1/\varepsilon$ **do**
  $i \leftarrow rand(\{1, \ldots, |w|\})$
  if $w[i] = a$ **then**
    $I_a \leftarrow I_a \cup \{i\}$
  if $w[i] = c$ **then**
    $I_c \leftarrow I_c \cup \{i\}$
if $\exists i \in I_a, j \in I_c$ s.t. $i < j$ **then**
  return No
return Yes

Example

$L = (a + b)^*(b + c)^*$ over $\Sigma = \{a, b, c\}$.



Minimal blocking factors $= cb^*a$.

$I_a, I_c \leftarrow \emptyset$
for $1 \leq k \leq 1/\varepsilon$ **do**
  $i \leftarrow rand(\{1, \ldots, |w|\})$
  if $w[i] = a$ **then**
    $I_a \leftarrow I_a \cup \{i\}$
  if $w[i] = c$ **then**
    $I_c \leftarrow I_c \cup \{i\}$
if $\exists i \in I_a, j \in I_c$ s.t. $i < j$ **then**
  return No
return Yes

Blocking sequence = Sequence of words that do not appear in that order in a word $w \in L$.

$$\mathcal{BS}(L) = \{(u_1, \cdots, u_k) \mid \Sigma^* u_1 \Sigma^* \cdots u_k \Sigma^* \cap L = \emptyset\}$$

Blocking sequence = Sequence of words that do not appear in that order in a word $w \in L$.

$$\mathcal{BS}(L) = \{(u_1, \cdots, u_k) \mid \Sigma^* u_1 \Sigma^* \cdots u_k \Sigma^* \cap L = \emptyset\}$$

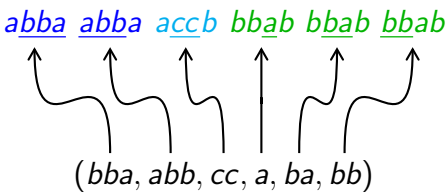
We use the order $\preceq$ such that
$(u_1, \ldots, u_k) \preceq (v_1, \ldots, v_r)$ iff $\exists i_1 \leq \cdots \leq i_k$ such that u_j is a factor of v_{i_j} for all j.

Blocking sequence = Sequence of words that do not appear in that order in a word $w \in L$.

$$\mathcal{BS}(L) = \{(u_1, \cdots, u_k) \mid \Sigma^* u_1 \Sigma^* \cdots u_k \Sigma^* \cap L = \emptyset\}$$

We use the order $\preceq$ such that
$(u_1, \ldots, u_k) \preceq (v_1, \ldots, v_r)$ iff $\exists i_1 \leq \cdots \leq i_k$ such that u_j is a factor of v_{i_j} for all j.

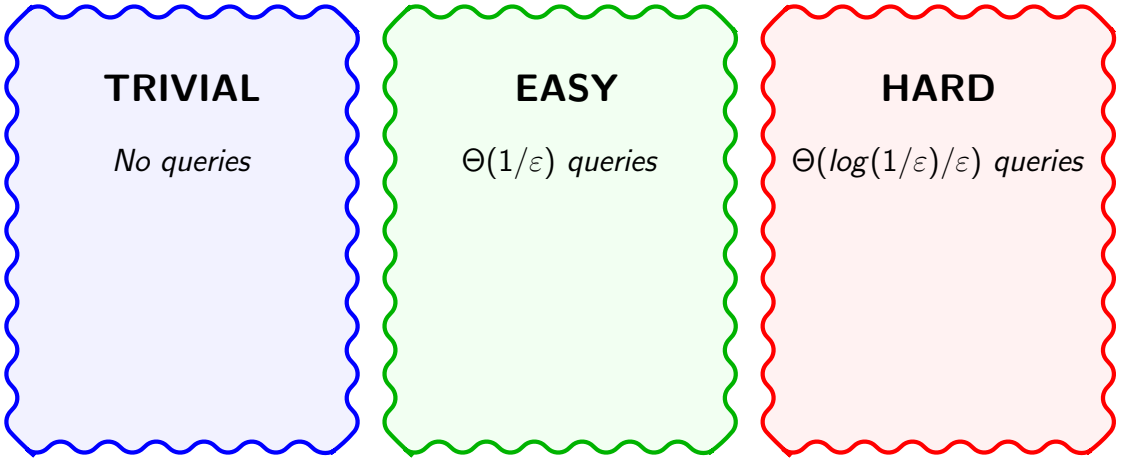
$(abba, accb, bbab)$



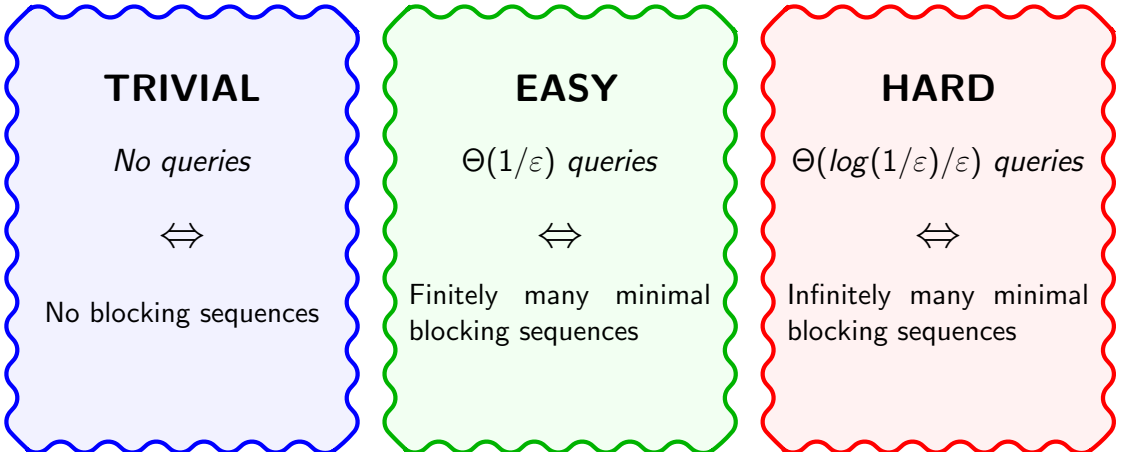
$\mathcal{MBS}(L)$ = minimal blocking sequences of L.

$$\mathcal{MBS}(a^* b^* c^*) = \{(b, a), (c, a), (c, b)\}$$

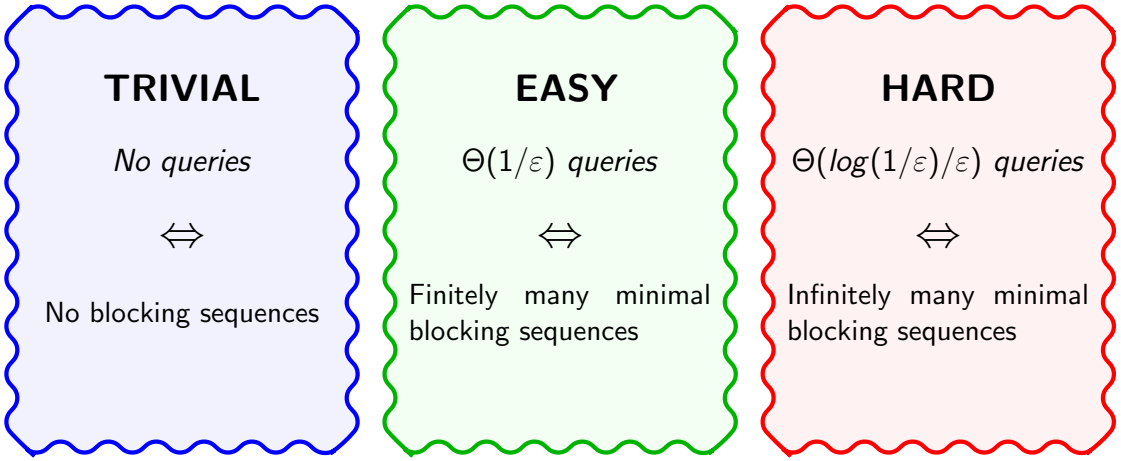
Trichotomy Theorem



Trichotomy Theorem

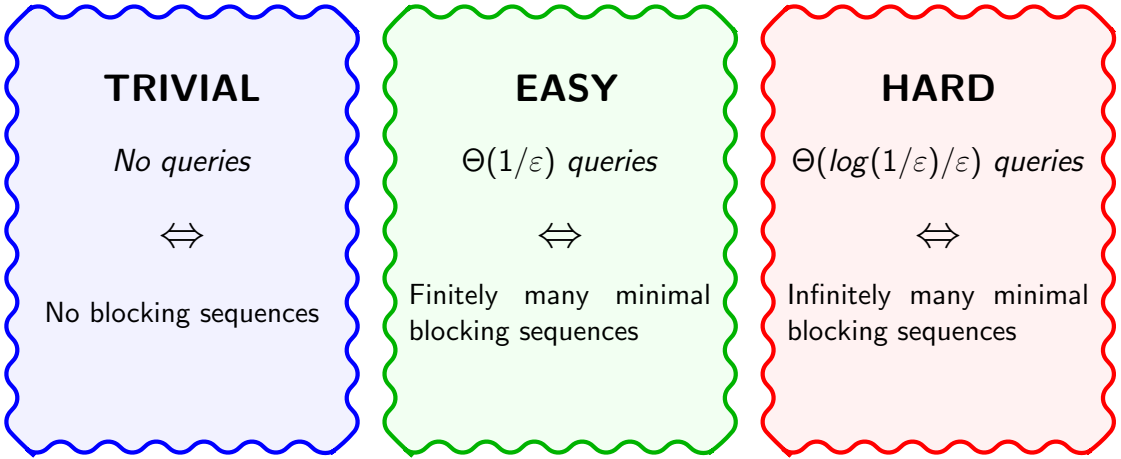


Trichotomy Theorem



Lower bounds obtained by reduction to the strongly connected case.

Trichotomy Theorem



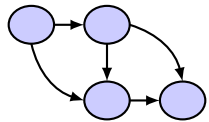
Lower bounds obtained by reduction to the strongly connected case.
If L is given as an NFA, classifying it is **PSPACE-complete**.

Lemma

w is far from $L(\mathcal{A})$ if and only if for all path of SCCs $S_1 \to S_2 \to \cdots \to S_k$ in $\mathcal{A}$, w contains many blocking factors for S_1, then many for S_2,...

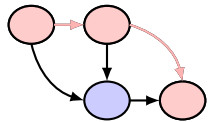
Lemma

w is far from $L(\mathcal{A})$ if and only if for all path of SCCs $S_1 \to S_2 \to \cdots \to S_k$ in $\mathcal{A}$, w contains many blocking factors for S_1, then many for S_2,...



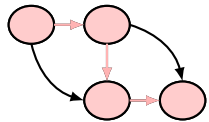
Lemma

w is far from $L(\mathcal{A})$ if and only if for all path of SCCs $S_1 \to S_2 \to \cdots \to S_k$ in $\mathcal{A}$, w contains many blocking factors for S_1, then many for S_2,...



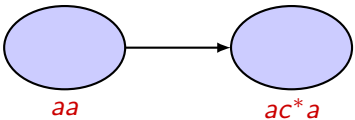
Lemma

w is far from $L(\mathcal{A})$ if and only if for all path of SCCs $S_1 \to S_2 \to \cdots \to S_k$ in $\mathcal{A}$, w contains many blocking factors for S_1, then many for S_2,...



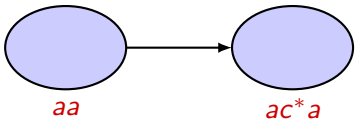
Lemma

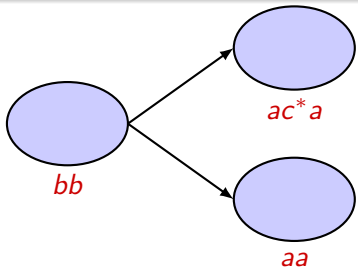
w is far from $L(\mathcal{A})$ if and only if for all path of SCCs $S_1 \to S_2 \to \cdots \to S_k$ in $\mathcal{A}$, w contains many blocking factors for S_1, then many for S_2,...



Lemma

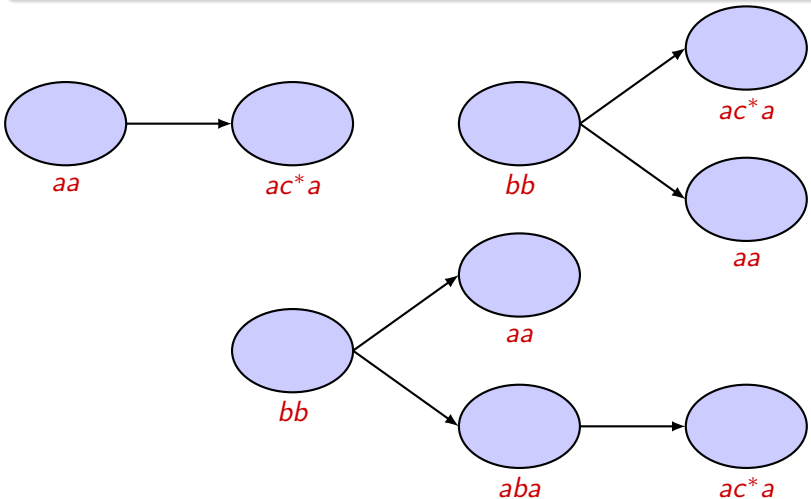
w is far from $L(\mathcal{A})$ if and only if for all path of SCCs $S_1 \to S_2 \to \cdots \to S_k$ in $\mathcal{A}$, w contains many blocking factors for S_1, then many for S_2,...





Lemma

w is far from $L(\mathcal{A})$ if and only if for all path of SCCs $S_1 \to S_2 \to \cdots \to S_k$ in $\mathcal{A}$, w contains many blocking factors for S_1, then many for S_2,...

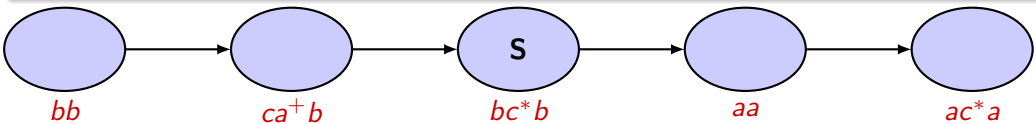


Theorem

An automaton has a hard language if and only if we can *isolate* a hard SCC.

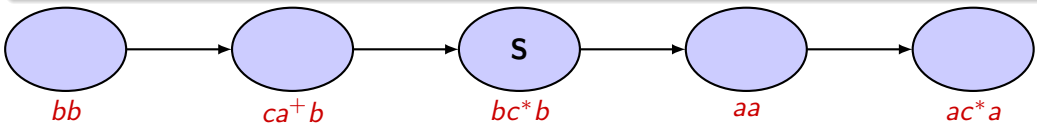
Theorem

An automaton has a hard language if and only if we can *isolate* a hard SCC.



Theorem

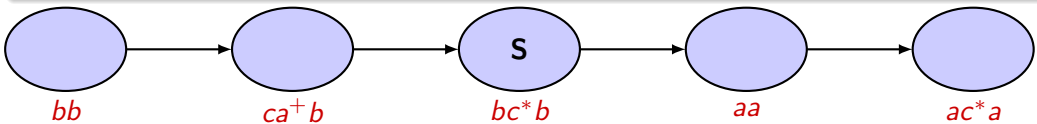
An automaton has a hard language if and only if we can *isolate* a hard SCC.



$(bb)^N(cab)^N w(aa)^N$ is far from $L(\mathcal{A})$ iff w is far from $L(\mathbf{S})$.

Theorem

An automaton has a hard language if and only if we can *isolate* a hard SCC.



$(bb)^N(cab)^N w(aa)^N$ is far from $L(\mathcal{A})$ iff w is far from $L(\mathbf{S})$.

Testing $L(\mathbf{S})$ is easier than testing $L(\mathcal{A})$.

What is left to do

- Edit distance, complexity for DFAs,...
- Which pushdown languages are testable?
- Tree automata / graphs of bounded treewidth?

What is left to do

- Edit distance, complexity for DFAs,...
- Which pushdown languages are testable?
- Tree automata / graphs of bounded treewidth?

Thank you for your attention!